

Lab 3 – Answers (Group X: Alaa Alqurashi, Faisal Alburti, Hashim Alsayed)

Task 1

harry.py – truth table ($\text{KB} = (\neg \text{rain} \rightarrow \text{hagrid}) \wedge (\text{hagrid} \vee \text{dumbledore}) \wedge \neg(\text{hagrid} \wedge \text{dumbledore}) \wedge \text{dumbledore}$)

rain	hagrid	dumbledore	KB
F	F	F	F
F	F	T	F ($\neg \text{rain} \rightarrow \text{hagrid}$ fails)
F	T	F	F (dumbledore is false)
F	T	T	F (both hagrid and dumbledore)
T	F	F	F
T	F	T	T
T	T	F	F
T	T	T	F

The KB is true in only one row, and rain is true in that row. Since rain holds in every model where the KB is true, `model_check` returns True. In words: dumbledore is true, so hagrid must be false (not both). Then $\neg \text{rain} \rightarrow \text{hagrid}$ can only hold if rain is true.

```
$ python3 harry.py # original
False
```

clue.py – output: MsScarlet: YES, library: YES, knife: YES

- MsScarlet: someone did it. Mustard is ruled out (initial cards) and Plum is ruled out (known card), so it must be Scarlet.
- library: Kitchen (initial) and ballroom (known) are ruled out, so it must be the library.
- knife: the unknown card says $\neg \text{scarlet} \vee \neg \text{library} \vee \neg \text{wrench}$. Scarlet and library are both true, so wrench must be false. Revolver is ruled out (initial), so it must be the knife.

```
$ python3 clue.py
MsScarlet: YES
library: YES
knife: YES
```

Task 2

harry.py without the fact `dumbledore`

- Prediction: False.
- Result: False.
- Why: The KB is now true in three rows: (rain=T, hagrid=T, dumbledore=F), (rain=F, hagrid=T, dumbledore=F), and (rain=T, hagrid=F, dumbledore=T). In one of them rain is false, so the KB no longer entails rain. That only shows rain can't be concluded, not that it's false.

```
$ python3 harry.py    # dumbledore removed
False
```

Modified harry.py:

```
from logic import *

rain = Symbol("rain")
hagrid = Symbol("hagrid")
dumbledore = Symbol("dumbledore")

knowledge = And(
    Implication(Not(rain), hagrid),
    Or(hagrid, dumbledore),
    Not(And(hagrid, dumbledore))
)

print(model_check(knowledge, rain))
```

puzzle.py without `MinervaGryffindor`

- Prediction: nothing can be placed.
- Result: nothing prints.
- Why: The only clues left are Gilderoy $\in$ {Gryffindor, Ravenclaw} and Pomona $\neq$ Slytherin. That leaves 8 valid assignments, and no person/house pair is true in all of them, so nobody can be placed.

- New clue: `knowledge.add(Symbol("HoraceGryffindor"))`. It gives a unique answer: Horace is in Gryffindor, so Gilderoy is in Ravenclaw. Pomona can't be in Slytherin, so she's in Hufflepuff, and Minerva is in Slytherin. Output: GilderoyRavenclaw, PomonaHufflepuff, MinervaSlytherin, HoraceGryffindor.

```
$ python3 puzzle.py # MinervaGryffindor removed
(no output - nobody can be placed)
```

```
$ python3 puzzle.py # new clue HoraceGryffindor
GilderoyRavenclaw
PomonaHufflepuff
MinervaSlytherin
HoraceGryffindor
```

Modified puzzle.py:

```
from logic import *

people = ["Gilderoy", "Pomona", "Minerva", "Horace"]
houses = ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]

symbols = []

knowledge = And()

for person in people:
    for house in houses:
        symbols.append(Symbol(f"{person}{house}"))

# Each person belongs to a house.
for person in people:
    knowledge.add(Or(
        Symbol(f"{person}Gryffindor"),
        Symbol(f"{person}Hufflepuff"),
        Symbol(f"{person}Ravenclaw"),
        Symbol(f"{person}Slytherin")
    ))

# Only one house per person.
for person in people:
    for h1 in houses:
        for h2 in houses:
            if h1 != h2:
                knowledge.add(
                    Implication(Symbol(f"{person}{h1}"), Not(Symbol(f"{person}{h2}"))))
        )
```

```

# Only one person per house.
for house in houses:
    for p1 in people:
        for p2 in people:
            if p1 != p2:
                knowledge.add(
                    Implication(Symbol(f"{p1}{house}"), Not(Symbol(f"{p2}{house}"))))
                )

knowledge.add(
    Or(Symbol("GilderoyGryffindor"), Symbol("GilderoyRavenclaw"))
)

knowledge.add(
    Not(Symbol("PomonaSlytherin"))
)

# Task 2: removed the clue MinervaGryffindor
# New clue (our own) that makes the answer unique again:
knowledge.add(
    Symbol("HoraceGryffindor")
)

for symbol in symbols:
    if model_check(knowledge, symbol):
        print(symbol)

```

Task 3 – knights.py output and reasoning

- Puzzle 0: A is a Knave. A knight can't say a contradiction, because it would have to be true. So A is a knave, and the statement is false, which is consistent.
- Puzzle 1: A is a Knave, B is a Knight. If A were a knight, "we are both knaves" would be true, making A a knave, which is a contradiction. So A is a knave and the statement is false. Since A is a knave, B can't be one, so B is a knight.
- Puzzle 2: A is a Knave, B is a Knight. If A were a knight, B would be the same kind (a knight), so B's claim "different kinds" would be true, which contradicts "same". So A is a knave, A's claim is false, and they are different kinds. That makes B a knight, and B's statement is true, which is consistent.
- Bonus Puzzle 3: A is a Knight, B is a Knave, C is a Knight. Nobody can say "I am a knave": a knight would be lying, and a knave would be telling the truth. So B's claim that A said it is false, which makes B a knave. Then "C is a knave" is false, so C is a knight. C says A is a knight, so A is a knight.

In the code, each character is exactly one of knight or knave. Each statement S by X is encoded as $\text{Implication}(X\text{Knight}, S) \wedge \text{Implication}(X\text{Knave}, \text{Not}(S))$.

knights.py:

```

# Lab 3 - Task 3: Knights & Knaves
from logic import *

```

```

AKnight, AKnave = Symbol("A is a Knight"), Symbol("A is a Knave")
BKnight, BKnave = Symbol("B is a Knight"), Symbol("B is a Knave")
CKnight, CKnave = Symbol("C is a Knight"), Symbol("C is a Knave")

def kind(knight, knave):
    # each character is exactly one of knight or knave
    return And(Or(knight, knave), Not(And(knight, knave)))

def says(knight, knave, statement):
    # a knight's statement is true, a knave's statement is false
    return And(Implication(knight, statement), Implication(knave, Not(statement)))

# Puzzle 0: A says "I am both a knight and a knave."
knowledge0 = And(
    kind(AKnight, AKnave),
    says(AKnight, AKnave, And(AKnight, AKnave))
)

# Puzzle 1: A says "We are both knaves." B says nothing.
knowledge1 = And(
    kind(AKnight, AKnave), kind(BKnight, BKnave),
    says(AKnight, AKnave, And(AKnave, BKnave))
)

# Puzzle 2: A says "We are the same kind." B says "We are of different kinds."
knowledge2 = And(
    kind(AKnight, AKnave), kind(BKnight, BKnave),
    says(AKnight, AKnave, Or(And(AKnight, BKnight), And(AKnave, BKnave))),
    says(BKnight, BKnave, Or(And(AKnight, BKnave), And(AKnave, BKnight)))
)

# Bonus Puzzle 3:
# A says either "I am a knight." or "I am a knave.", but you don't know which.
# B says "A said 'I am a knave'." B says "C is a knave." C says "A is a knight."
knowledge3 = And(
    kind(AKnight, AKnave), kind(BKnight, BKnave), kind(CKnight, CKnave),
    # A said one of the two sentences (exclusive)
    Or(
        And(says(AKnight, AKnave, AKnight), Not(says(AKnight, AKnave, AKnave))),
        And(says(AKnight, AKnave, AKnave), Not(says(AKnight, AKnave, AKnight)))
    ),
    # B: "A said 'I am a knave'"
    says(BKnight, BKnave, says(AKnight, AKnave, AKnave)),
    says(BKnight, BKnave, CKnave),
    says(CKnight, CKnave, AKnight)
)

def main():
    symbols = [AKnight, AKnave, BKnight, BKnave, CKnight, CKnave]
    puzzles = [("Puzzle 0", knowledge0), ("Puzzle 1", knowledge1),
               ("Puzzle 2", knowledge2), ("Puzzle 3", knowledge3)]
    for name, knowledge in puzzles:
        print(name)
        for symbol in symbols:
            if model_check(knowledge, symbol):
                print(f"    {symbol}")

```

```
if __name__ == "__main__":  
    main()
```

```
$ python3 knights.py
```

```
Puzzle 0
```

```
    A is a Knave
```

```
Puzzle 1
```

```
    A is a Knave
```

```
    B is a Knight
```

```
Puzzle 2
```

```
    A is a Knave
```

```
    B is a Knight
```

```
Puzzle 3
```

```
    A is a Knight
```

```
    B is a Knave
```

```
    C is a Knight
```